ActInf ModelStream #001.2: "A Step-by-Step Tutorial on Active Inference"

Source: YouTube **Playlist**: ModelStreams **Duration**: 2:25:11 | **Uploaded**: Unknown **Transcript method**: auto_caption

---

hello everyone welcome to the active inference lab and to the model stream 2.0 today is going to be a really awesome discussion so let's just introduce ourselves and get right into it i'm daniel friedman i'm a postdoctoral researcher in davis california ryan um yep so i'm ryan smith i'm an associate investigator at the uh laureate institute for brain research hi i'm christopher i'm a phd student at the university of cambridge and i'm max murphy and i'll be starting my post-doctoral fellowship at carnegie mellon in hopefully a week awesome thanks to ryan and christopher the authors uh or two of the three authors of this paper for joining today and for max for lending your expertise as well so this is the second model stream in a multi-part series that is going to be highlighting perspectives and addressing questions related to applied active inference and so right now we're looking at the tutorial paper of smith at all a step-by-step tutorial on active inference and its application to empirical data the link uh you can find in the video description so if you have any questions or thoughts or comments during the live stream type it in the youtube live chat and we'll be monitoring that and then if you have any questions after the live stream leave a comment on the video or get in touch with us another way and we'll try to address it in a future session and if you want to learn more or if you want to participate go to activeinference.org all right so today in our second session we're going to be picking up with a brief overview and summary of where we're at where we've been we're going to go through a few clarifying questions that have been raised in the last week and then we're going to dive into the specific examples and into the code and do some code walk through so that will be really exciting all right so i'll just throw out the first opening question which is who is this session for what kinds of tools and skills would you say are required and which kinds of tools or skills might be helpful um okay well um i mean i would say that the session and i mean really the entire tutorial is for people who um are just kind of i mean mainly people who are just starting out people who want to be able who are interested in um but maybe don't have a lot of background in mathematics you know especially like people

coming from like neuroscience and psychology backgrounds for example that want to or people who do some computational modeling but like maybe like simpler reinforcement learning models things like that where active inference is kind of notoriously a little bit more kind of like black box you know it's harder to kind of break into um a lot of people think that the models are described in a way that's kind of like opaque um you know and so the the idea was to give people the background needed um and the actual example code um that would be needed that could be adapted to build your own models do your own simulation studies and apply and build models of task studies and be able to fit models to data in other words to kind of start to finish to be able for a person who wants to to be able to use active inference models to model data in empirical studies and actually use them in their own research so as far as tools necessary um you know the main thing right now is going to be having some minimal understanding or motivation to learn how to use matlab you know so you know we in the um tutorial code so i mean we have like seven or eight different you know tutorial scripts you know this point but the main one is um the one that i'll be focusing on today and it's very heavily commented um so it kind of best we did try to describe kind of like line by line each thing that's going on i mean it's honestly there's enough comments and there'd almost be kind of like a second paper in and of itself um so um you know we're hoping um that even someone who doesn't have a ton of background in matlab will be able to kind of bootstrap their way up um but you definitely need to have a little bit or be motivated to learn at least a little bit anything else to add on that christopher or max no i think that sums it up well just one note on the code there so you mentioned it's in matlab now what about other programming languages or backgrounds like python so what other languages are people in the active inference space developing in and how could we include more computer languages as well as natural human languages so i mean there are so at present everything is almost exclusively in matlab unfortunately and the reason for that is is that a lot of this is actually built off of spm scripts you know that carl fristen originally wrote in relation to uh neural imaging approaches so things like dynamic puzzle modeling and so a lot of a lot of the current scripts for doing active inference call on a broad range of other spm scripts that are used for a lot of different purposes for instance in sort of dealing with large matrices in like neuroimaging so that so for that reason and because carl is primarily worked in matlab and relation spm everything has kind of been in spm and specifically in what's called the dem toolbox in spm12 stands for dynamic expectation maximization there's a ton of active inference scripts as well as a ton of other modeling scripts for say like continuous models and other sorts of things in there and we just

make use of a couple of those um the main ones not the ones they call on but um but uh that being said there are people who are trying at the moment to um translate some of the stuff into python uh alec schatz for example um is currently in the process of trying to put together uh python versions of some of the main simulation and inversion scripts um i don't i mean they're very kind of in beta at the moment as i understand them but i know i know that that's something that's in progress cool so it sounds like because this project has such a history of being developed in matlab and the spm toolkit and carl fristan's research and collaborations were um drawing heavily on matlab however in the modern development context there's a lot of open source projects that are developing in python so that's definitely an area we'll hope to be expanding and also a great opportunity for people who are experienced in python or matlab and want to be curious about this is the opening to help contribute to something that could be really exciting so just a last um last uh warm-up introduction question what kinds of projects or questions or investigations do you think this could apply to for somebody who they've heard of scientific modeling but not active inference modeling what do you think are some low-hanging fruit or really straightforward interpretations where you think it could be applied so i mean i guess there's a couple questions here i mean one is um you know like well i guess the main one is given that the kind of history of computational modeling has um largely um than in the um like reinforcement learning i mean a sort of literature or i mean there are others like drift of fusion modeling and things like that but um um the you know one of the kind of major questions is oh well active inference models seem like they're really complicated you know relative to some of these simpler uh sort of traditional reinforcement learning models um so what is it that active inference actually adds right like what can you do using an active inference model in a study that um you maybe couldn't do or could do as naturally using standard sorts of existing reinforcement learning models and you know here the the kind of answer that i would want to give is there's a few parts to this one is that um because the kind of value or cost function in active inference is expected for energy as opposed to a reward function and because the expected free energy has a reward a reward component and an information value component then it's it's quite a bit more natural in active inference models to model explore exploit tasks you know so tasks where the agent has to somehow intrinsically assign value to seeking information as a means of knowing how to maximize reward eventually so there are and there are actually two different types of um sort of directed or goal driven explanation or exploration in active inference one is um which is called parameter exploration and the idea here is this is probably the closest thing to um reinforcement learning

where say the agent might not start out knowing what the reward probabilities are if they choose one action versus another so they will choose actions that will help them essentially learn best what the reward probabilities are before you know just reliably you're continuously selecting the same option right because that's the kind of know which one's the most rewarding first so that's called parameter exploration and that that like i said has analogs in reinforcement learning but doesn't have like a specific in in standard models anyway it doesn't have a very specific sort of information drive um to uh to seek that out um but there are some so things like information bonus terms uh there's different things like that in reinforcement learning that can do something like that um but they're kind of tacked on their little little they have a little more kind of like ad hoc feeling to them i guess um whereas it kind of emerges naturally from expected free energy in active inference um there's also something that's a little more unique in active inference which is called state exploration and that's more where you're just kind of choosing to move to states that you expect will give you the most precise observations about what state you're in um you know so this would be the thing like you don't know whether you're in the green room or the red room so you should go turn on a light so you'll know which which room you're in um right so that also falls out of expected free energy very naturally but not out of a lot of other traditional computational modeling approaches um the uh there's also sorry i could go out for a while about this but there's also something in reinforcement learning models that has to do with explore exploit called random where people essentially start behaving more randomly if they're uncertain if they know they have to make a bunch more future choices but those values are typically fixed in whereas the uh expected free energy precision uh parameter and active inference is actually a dynamically updated version of that so it changes to remain driving essentially the amount of randomness and behavior that's um expected given past observations um and so that's kind of nice um the last thing i would say is is that active inference has a very kind of specific neural process theory associated with it so it makes very specific predictions about the neural responses you ought to see in studies whereas most other models don't so those are that's what i would say thanks for the answer any thoughts on that or any of the questions christopher or max i would just say very broadly speaking active inference is really great for modeling decision-making under uncertainty and in particular perceptual various types of perceptual decision making and indiscreet state spaces so you can do anything from modeling so like ryan and i have some previous work where we show that you can reproduce a whole lot of kind of classic visual awareness neural correlates and like the classic visual awareness paradigms for example using a hierarchical model and

the reason we can do that is because it had active inference has such a detailed and rich process theory attached to it and so for me i think what's really unique about it is that so there's lots of different ways of providing unified explanations but one way that you might want to provide a unifying explanation is just have a unifying set of kind of uh com unifying computation or one computational architecture where you can draw in all of these different things into one paradigm and then understand or well one computational paradigm that is then understand how they're related through the different parameters that are being modulated when you or being fit when you build models for so i'm not aware of any other modeling paradigm really where you could model things as diverse as a basic kind of explore exploit task or looking at kind of phasic dopamine dynamics and then something that goes all the way up to something like visual awareness or even like planning and bellman optimal planning so to me that's what's so special about this if you if you want to unify things under one umbrella this is really um yeah one one last thing i'll say and this is just kind of a practical nice thing is this is that the in most other um modeling approaches you actually have to kind of change the equations you know like to fit it to a specific um you know task whereas an active inference technically you don't all you do is the the actual update equations are always identical all you have to do is write down the appropriate generative model um so it's nice because it's really generic and and you know once you have the generative model specified everything else kind of just comes for free and just to jump in off of that uh generalizability the the reason i'm interested i'm somebody who studies movement a totally different area of neuroscience than decision making and uncertainty modeling but at the same time this framework what kind of interests me about it is that exactly that which is that it seems that i can start out with kind of a structure generative model that is consistent even across domains and then potentially use that to to apply it to my own neuroscientific um the the main questions that i have and things that i'll be interested to hear about moving forward would be um you know we talked about that we can use it both for parameter exploration and state exploration but when we're in in the empirical context of the experiments being performed for um it might be the case that i'm uncertain about certain parameters or i'm uncertain about the state and i may be uncertain about both of those things simultaneously so um finding a way to either design my experiments so that i impose certainty on either the state or the parameters is probably seems to me an important part of using this model no no it's a great point actually me and chris we're just talking about this like like yesterday or something right like how to design an optimal task to disambiguate but include both having to solve problems that involve state and parameter

exploration simultaneously technically the example model that we have for the tutorial is able to do that um but not in an incredibly interesting way uh justin justin kind of way to show that how you could do it very interesting about the experimental design because the kind of statistical techniques like a t-test we know the statistical power calculations or we might know the kinds of experimental designs that lead to good clean t-tests but it's a great point about the experimental design and just to highlight this generalizable and also generative nature we talked about a lot of different areas of research a lot of different frameworks but the parts that stood out to me were christopher you said decision making under uncertainty and action and perception that covers a lot a lot of different areas of neuroscience and then we are looking for this kind of unified explanation but rather than an explanation which might be philosophical or based upon how much we understand it or how we feel that one day we're rather going to look for a unified computational architecture or a computational paradigm as you said that will let us compare and design and develop these generative and generalizable models so really cool discussion and i'm just going to ask one question from the youtube live chat then we'll go to ryan with a screen share cb asks can active inference research help us understand the neuroscience of stimulus independent thoughts when there isn't any sensory input at all thanks for elaboration in this regard i mean i guess i mean the broader answer would say yes but um the way that you would do it would probably be pretty task specific um you know so so i'm um i'm not necessarily sure how to give a really precise detailed answer without knowing the particular like task or even just general psychological that's being imagined um so i mean it's actually really easy to model for instance like working memory tasks where you might get a stimulus at one point but then there's no stimulus for a really long time and somehow the thing has to continue to maintain representations um in the absence of any stimulus to know what to do you know at some distant future point um um you know just there's that kind of thing um i mean just as one example i again i like i said it the way to answer the question is probably is it would be different depending on the exact sort of psychological or task context in question cool so just like explore exploit there are tasks that looked more exploratory or more exploitative almost within this action of perception we're going to be talking about some tasks that are more towards the action-oriented side like the policy planning or the motor behavior and other tasks that are more towards almost the perceptive side like this visual awareness task now ocular motor movement the movement of the eye muscles is still a behavior and there's papers that model that but it's almost like we're looking at the whole spectrum of explore exploit or action perception reinforcement learning and wondering how we can again make

this unified framework for talking about these different kinds of systems so shall we jump right in ryan or go ahead anything else well i mean unless i mean just just one last point to bring up in relation to their you know this person's question is just that it's actually what's really important and probably a more recent development and this is stuff that um some of the visual awareness stuff that me and chris have done and also um some stuff uh that i've done previously with like emotional awareness for example like models of emotional awareness um there's there's a really important sense of what you're doing not just behavioral actions but actually cognitive actions um right so you're selecting policies that just do something like in the neural implementation just do something like change the the functional connectivity between two brain regions or or change in other words it is stimulus independent uh action they're actions that that have to do with changing what you're doing cognitively and not what you're doing behaviorally um you know so that's also an important uh part of it that's been done in you know a few previous papers that um i uh relates an important way to this kind of stimulus independence sort of question um but again only if i understood it right cool good answer and thanks for that so go ahead and share your screen and just however we can help whatever you want us to do or not do we're there but i'm really looking forward to this okay so the um you know the main the main thing here that i want to do just to start as you know and this is in part just because i'm assuming that not everyone who's watching today was watching uh last week um so i'm not going to be able to give you a full understanding of where we're sort of of how we're segueing from last week to this week but i just want to give kind of like a really sort of uh rapid overview of where we're kind of going um you know so first we covered this slide here which just um shows an example of how you can do um how you can do perception as bayesian and then and this is just an example a mathematical example of how it works um but the idea is where you're you see this sort of two-dimensional image and you're this gray thing under great disc under observation you're trying to infer based on prior expectations whether it's a three dimensionally it's a concave or convex shape and so the idea is is that in simple problems you can do that exactly by solving um bayes theorem so this equation up here on the upper left but in most cases that's actually intractable to do and so what you do in active inference is you instead um define this thing f here which is called expected free energy um and it's more or less a measure of the accuracy of predictions minus the accuracy of beliefs and combined with the complexity of those beliefs essentially how much you have to change your beliefs to to get a model that's accurate so by finding a set of beliefs that minimize expected free energy in other words ones that are accurate while

changing as little as possible um this is just an example of how by doing that by just kind of searching around for different qs's here different beliefs about the probability of states then you can find one that when you calculate it gives the lowest value for f and that's going to be the one that's as close as possible to the true uh answer the bayes theorem would give you um and so uh so what you're um i'll kind of skip this one but well so what you're trying to do then an active inference is you have some true thing going on out in the world true states and the observations that they generate um but then the brain has this model where it's trying to say okay what state must i be in given that i got this observation so the brain is trying to essentially come up with the representation that best matches what's going on in the generative process in other words the thing that's truly generating the observations um and so that's probability of o and s so states and observations this pi thing is policies so it's what's the most likely action i ought to choose given that the world is this way and given that i prefer some observations over others and what's going to be important for what i cover today is the way that you actually set up these generative models and the way that these are set up is you have these states s and you have these observations o um and then you have prior initial prior expectations that we just called d um and if you solve this right if you get some observations and you say given my generative model what states are most likely given those observations and given my prior expectations d then you get your sort of optimal belief about what the what the states ought to be given those observations but in active inference we do this in a way that's also dynamic so for instance states at one at time one are going to transition into states at time two and so you also have beliefs this thing in b about what states are most likely to transition into what other states um and then um and this is kind of one of the unique moves in active inference is that policies so the different action sequences you might choose are just models as things that entail different transitions between states so b matrix 1 for example here would entail choosing to move from one state to another state whereas some different policy would entail a different b matrix that means you transition from one state to some other state and the choice of policy depends on g here which is the expected free energy which is a function of c which is your preferences so essentially what you're trying to do is you're trying to find the policy which corresponds to a set of state transitions that will generate the observations over that are going to be as consistent as possible with uh your preferences which corresponds to having the lowest g um and so and then there's some other sort of parameters up here that can say add habits so e here can give you kind of an additional prior over policies gives you kind of habits to do one thing over another it can have this beta gamma thing that

also controls how uh precise or how reliable um the model expects uh the expected free energy estimates to be um so i realize that's a lot um but we kind of covered this more slowly last time but what you really need to know um when we're actually putting together the code like i'm going to walk you through this is what we're going to be specifying step by step we're going to be specifying what the observations are um we're going to be specifying the likelihood a so in other words what states generate what observations with what probability we're going to be specifying d and b which are the priors of our initial states and beliefs about how states are going to transition um and we're going to define policies this pi thing which is going to tell us the different possible state transitions we could choose um and then we're going to find c here which is what the agent's preferences are what observations it wants to get over others um as well as the habits and this beta thing the expected free energy precision so building a model in the code amounts to just specifying what all these things are what the state space is what the observation space looks like what a and d and b look and what c looks like and maybe e and beta and so if you want to know the exact equations for solving this um you know we've shown them here but again i'm not gonna describe them in detail um but basically all they're doing is they're saying given my beliefs about state transitions so my priors and given a my likelihood what's my most likely posterior belief going to be which is just bayesian inference right priors and likelihoods together equal posteriors um or are approximate to posteriors technically um but so that's what we're doing we're going to make one comment on that slide ryan yeah yes so on the top left we have state inference given priors and given observations so that's kind of inference and we're going to be building on the strata to go from inference bayesian inference to active inference so you go from the top left which is just perception at one moment in time that's static to the bottom left where you have states that are updating through time given priors and given observations you're updating your estimate through then you go to the top right so that so the bottom left square adds in time now on the top right we add in this element of policy selection or control theory and then on the bottom right we get all the way up to this full active inference model which not just has a control theory element through policy but some extra parameters that help us as we're going to find out soon like e and beta so just to give that one more time because this is what's unique about and this is the skeleton of the model that everything else is going to be scaffolded onto thanks ryan continue yeah no thank you that was definitely a good uh quicker uh clear summary um so appreciate it um okay so and then again just to remind people the end goal of all of this is to learn how to put together um particular generative models right particular sets of a matrices d vectors b

matrices c matrices etc for empirical so for behavioral tasks that participants can do and then you can fit these models to actual behavior to learn things about participants for instance you might learn what their parameters are for a or for b or for instance what their beta value is um you know etc um so that's the that's the kind of ultimate goal here but the kind of first step to do that beyond just understanding the structure i was talking about is to know how to actually put that together in the code and so last time you know i gave a bunch of examples of past experiments that we and others have done where we have built active inference models for tasks fit them to data and learn things for instance about differences in substance use and learning rates for example or differences in beta and decision uncertainty in between healthy people and people with depression and anxiety or substance use um or differences in um perception of uh precision of precision estimates for uh interceptive perception for instance um you know so so i just kind of went through these examples of specific tasks and empirical results just to show the kind of thing that you can do with this in practice and but um and then the the task that we're modeling is one that's designed to be really simple but that also kind of showcases um the major important elements of active inference models um and so here what we're doing is we have a task where the agent starts in a start state and they can either directly choose to pull the left to choose the left slot machine or the right slot machine and if they do that they can either win four dollars or zero dollars depending on which machine is better the left one or the right one but they don't know ahead of time which one's better so they can either kind of take a risky guess and either get four dollars or zero or they can choose to seek information first to ask for this hint and the hint will tell them which uh the left one which of the left one of the right one is better on that trial um and if it's the left better context that means the left one will win 80 of the time and if the hint says the right one's better then the right one will pay out 80 of the time but there's a cost to taking the hint which is that if you take the hint and then choose a machine then you can only win two dollars instead of four um so that's what we're going to be modeling um and so what we need to specify for that is we need to specify that there are two types of hidden states one is whether it's the left better or the right better context whether it's a left better or a better trial and also the behavior states so choosing to move from the start state to the hint state or choosing to move from the start state directly to the left or right slot machine so that's two types of states um we have to specify the observations where in this case the major observations are winning the four dollars two dollars or zero dollars um or in this case we're actually just gonna model you win or you don't win and i'll show you how that works um and um and we'll have to define uh the agent's preferences so

essentially saying it prefers four dollars more than two dollars and two dollars more than zero dollars and that's going to be that c thing um so uh so that's kind of the general task structure and i went through um you know the way to do this kind of simply outside of the code which is you define your initial state priors you define your state outcome mapping you define your preference outcome preferences over outcomes you define the state transitions slash actions um so b and then you define the policies which you define as this letter of v um and then the final thing i'll talk about before going into the code is that in these models you can separate out the generative process so it's actually out in the world generating observations from the generative model which is the agent's beliefs about what's generating the observations um and if we want to separate those in then what we do is big d or big a um or big b um those are the generative process um but if you also specify a little d or a little a or a little b then that's the generative model so that's the agent's beliefs and if you specify those then they will be separate from the generative process and that's typically what you need to do that is what you need to do if you want to model learning which uh i'll show ryan just just two two quick points so first off um someone has asked will these slides be available or is there any resource where we can look at these slides because there's a lot of great information on them um i can make them available everything everything here is i mean almost all the figures that i'm showing um other than the actual task figure here are in the are in the tutorial paper and um everything i'm showing about how to actually build the models is is in the code and well explained in the code like i'll show um but i'm also happy to provide you know these slides i mean just just let me know um like i said the vast majority of them are literally just the figures in the paper um so uh so yeah i mean like i said i'm happy to make them available or send them um to anyone i could even just put them as supplementary additional supplementary material at the tutorial as well so i mean yeah just whatever whatever people want great idea and what a fun and accessible science we can have when we just receive questions like that and can put up our slides and then i think max there was a question that you had about the b matrix just while we're defining our terms what would you like to say about all right so uh but just real quick uh for anybody who's just jumping in on the stream the link is is below the video you can go to their paper if you're looking for that supplementary code it's in the supplementary materials you can download all of the code from their supplementary materials there the question i had was um one thing to touch on is as we walk through the code one thing i noticed is that we have kind of a tensorial structure for these matrices um where we could also specify a transition matrix that was that had like a 2d with you know d with with probability of 0.8 0.2

but instead we have a tensorial structure where we have a fixed probability of one for transitioning from any given state to another state and i just thought uh if you wanted to touch on maybe the motivation for that i i believe it's related to how the model like basically interpreting the model and and how the model gets used um is that uh i don't know if you wanted to elaborate on that any yeah i mean i mean yes i mean so technically you know we call these things like that we call like a matrix to the b matrix matrices but technically they're tensors um so they're high dimensional matrices and um so so and you know obviously go through this when we go through the code but because you can have different um types right so the state of being in the left better or the right better context versus the type of state that has to do with you know where you move to or what you choose right the being in a start state versus the hint state versus you know the left or right bandit or yeah slot machine choice state um because you have two different types of states those are called state factors and so essentially what you need to do is you need to specify these matrices or like i said really tensors that again this is for a that say um what outcomes for each type of outcome will be generated for each combination of each value for each state factor um you know which makes which makes it you know higher dimensional um which is probably the trickiest part i'll explain this as well as i as i can when we go through the code um but so but so that's that's that and also so simultaneously because you have two um types of states here that also means you have to have a b matrix a transition matrix for each um type of state um and when you when the agent doesn't have control about state about transitions for a certain state um then there's only going to be one of these uh b matrices um so for example here so this just says b for factor one which is what context you're in um this is just a simple uh 2d matrix um although technically for reasons in the code you define it as a as a three-dimensional thing with just a one here this is just but that's just because the third dimension here defines the the number label for the action um so this is basically just saying there's only one action for state factor one and that is an identity matrix here which just means that for each trial it's gonna be the left better context through the whole trial right in other words within a trial it's not gonna like after they take the hint it's not gonna like switch on them you know so really the right one is better if they got the hint that the left one was better this is just saying if you start in the left better one you're going to stay in the left better one the whole trial and that's it whereas for stay factor two so the b matrices for state factor two um there are four of these which correspond to four different possible um so here the third dimension says this is action one which means that from any state which are the columns you can move to state 1 which is the first row whereas

the second one says if i pick action 2 so third dimension value 2 then i can move from any of the four states again columns 2 state 2 which is the hint state and so forth for action 3 and action 4 moving to choose the left bandit or the left machine or choosing the right machine so these are going to be the four actions which are just the four possible um and then when you specify policies this thing in v this is just saying for which of those actions can i string together um so v for one here for state factor one is just a bunch of ones because again there's only one action it just stays in whatever state it started in um and each column here is a policy and each row is a time is a an action time point basically so this is saying so the first row here is what action could i take when moving from time one to time two and the third row and the second row here is what action could i take when moving from time to time three and again each of these columns just says stay in ones because there is no possible action for uh changing the context but policy the policy for a space for data or for factor two here um so again the the um there's these five columns and this just says i can move to the start state and i can stay in the start state right so that's just action one up here twice um then the second column here says i could take action two so look at the hint and then yeah and then uh move to action three which is choose the left machine or i could take the hint two and then choose four which is move to the right machine or i can go straight to the right machine and then go back to start or i can go straight to the or straight to the sorry three is the left machine or straight to the right machine action four and then go back to start so so in other words you know there's a bunch of other possible transitions right strings that you might think of right you might for instance go to the left machine and then go take the hint or something like that but we're not allowing that we're saying these are the allowable transitions so these are the different policies or action sequences that can be chosen so correct me if i'm wrong but it seems that this tensorial structure kind of allows us to compress all those different combinatorial you know combinations that we might have so it allows the structure of basically just allows the code to be a little more efficient is that at the end of the day yeah that's a perfectly reasonable way to talk about it um so so okay so so that already gives you kind of a little bit of a start here um so now um um and then and then in so at the end of the day what we're going to be doing is we're going to be specifying each of these things so t the amount of time steps v the policy space a the likelihood the state outcome mapping b the transition probabilities or transition matrices c which is the preferred uh observations this is preferred states here but it should be preferred observations um d it would be the priors of our initial states um and then all these other extra little parameters that you can or cannot include i'll talk about those later um but then we also

specify this little d which basically says the thing doesn't start out knowing the context so it has to learn the context over trials it has to sort of build up beliefs about d over trials um so with that as a kind of quick background i'm going to now kind of move into the code it's cool that the whole thing is stored as one object with a bunch of fields because it's almost like you could have ensembles of these different models or you might have a bunch of ants in a colony it really lends itself to being reproducible and transparent with what each parameter is and comparing a whole host of them alongside or different kinds of models just really cool to see that instead of just a tangle of parameters that you can get lost in this is like a column of things that each has a very clear purpose uh yeah so i should say so you you do yeah put all these things into the structure that would usually just call little mvp which just stands for markov decision um and then when you actually run the thing then you run it through the uh the uh vb the spm mdp vb underscore x uh code and we've provided a version that just appends it with the word tutorial um and you make that big mdp and then you can take use sub commands like this that just um generate figures that will show you the outcomes of the the simulation um and these are examples sort of simulation results but again i'll go into these after um we go through some of the code because to know how to actually use this stuff you need to understand the code um so hopefully i can make this not too monotonous you know walking through code isn't always the most exciting but um the way that we have this set up um is you know like i said we have comments that kind of try our best to explain what every little command means you know like what clear and close all mean what uh rng default means you know etc um i'm actually going to change this to shuffle and what that means is just that um the matlab will use a random number generator that um won't do the exact same thing each time so we can get variable results when we run it so we've provided um several different simulation options that we can use after building the model below which is what i'll walk through so if you set the sim variable equal to then that will just simulate a single trial which will reproduce uh figure seven um in in the paper um so that'd just be a single trial simulation if you set sim equal to two then that'll simulate multiple trials um where the left context is always active but um the agent won't know that right the agent has to learn okay the the left one is the one it always is so after a while it'll be confident and won't need to take the hint anymore um if i could just briefly interject there so that would be for example an example of what we talked about earlier as parameter exploration yes uh well well not exactly it would be an example of learning the parameters over successive um yeah so state i mean state exploration is a bigger thing here right because either they choose to move to the queue or not

to learn which context it is yeah i agree that so yeah the tricky thing is is that when you only have two options then parameter exploration would just amount to even though i've gotten some good results from choosing the left one i might also instead try to choose the right one a few times to be more confident in what the reward probability is for the right one but typically to get good dynamics like that you're probably gonna need something that has um has more like three arms like three different options instead of two if you're also going to include the state exploration thing with the q um it would get a little more complicated like a standard kind of paradigm for doing parameter exploration would be something where you don't have the queue and the um the agent just has like three choices and it just kind of has to choose different ones over time until it becomes confident which one is the best and then it'll just keep choosing that one so the parameter exploration would amount to how many times kind of it it chooses each slot machine initially just to figure out which one is best um and um a standard reinforcement learning agent will do that differently than an active inference agent um and the the substance use study i mentioned above actually specifically did that um but uh but yeah so just one question in this sim equals one two three four five they're like scenarios that you're going to be able to rapidly plug and play so scenario one is like a static or scenario two is multi-trial so this is what is going to allow reproducible construction of the figures and understanding of how different parameter changes influence behavior but that's what we're laying out right now it's kind of like the scenarios for the agent right okay yeah i don't know about it quick is one thing i think sim five would take about half an hour to run yeah don't do it sim five we'll talk about with when we talk about parameter fitting which will be a different session um but mainly we'll be talking about sim one two and three here which just has to do with perception and learning as opposed to the real empirical stuff which is estimating parameters um and uh you know using them you know for group level analyses and things like that well it's just so helpful though how it's laid out from the most static straightforward perceptive inference to action in the loop and then now that action's in the loop we're gonna think about the simplest setting to explore action in a loop on a single trial unchanging and then we're going to keep on adding layers of ecological variability or other features into the model so thanks for making it so step wise and that's why it's a step-by-step tutorial yeah yeah we tried um but that's the last thing for for today so if you set sem equal to three then um that will simulate a reversal learning uh situation and that's where basically the left context is active for a certain number of trials and so the agent should build up a prior expectation that it's just the left one every time so it'll stop taking the hint and just start choosing left but then it

switches and so now the right context is the right one is the correct one and you see basically how long it takes before the agent again says okay now i need to start taking the hint again and then eventually when it becomes confident to just choosing the right one you know directly um and not needing to take the hint um so it's kind of like a standard reversal learning kind of task um so those are the three that hopefully we'll get through today um and um there are several parameters you know that can be kind of messed with but the one that i've focused on mainly here at least we can set in the beginning is um this rs1 thing which just turns into rs later down um but that stands for risk seeking or it could also be reward seeking um but more or less the bigger this value is the stronger the agent's preference is for uh winning as much money as possible so it's this is the value that's in the c matrix that defines preferences over winning versus losing um so the bigger this number the the more the agent is going to say okay i care more about reward than about information and therefore i'm just going to take the risky choice to go for one of the slot machines directly so the the higher this the less information seeking the more the lower this is the more the more information seeking the agent will be um and so that's just kind of a really simple one to show how by tweaking parameters you can get different results here just allowing you to weight how strong the reward value component is in the expected free energy versus how strong the weight is of the information seeking component of the expected free um and then um you can kind of ignore this pebb thing down here this is basically you can turn this on if when you do sim equals five you want to actually do group level bayesian analyses on the parameters um this will also save the outputs of sim like chris said sim 5 like takes like a half hour to run so if you turn this thing on it'll also save the results of sim 5 so you can run this peb thing without having to reduce m5 every time so we just figured hopefully that would help i just i just want to jump in and point out that it's really nice in this part where for example with ourselves set there now let's say that i'm designing my experiment and i need to know how many trials and how many uh subjects do i need when i'm doing my power analysis i can run a simulation like this set that parameter at what i think might be a reasonable range of values from the literature and then i can say well this is the expected difference in my groups based on what i think this range of values might be so it's a really powerful tool to be able to plug and play a single parameter like that yeah i know it's nice i mean typically with experimental you know with experiments what you do want to do first is you want to kind of see hey can tweaking different parameters in the model produce simulated behavior that looks a lot like the type of behavior i actually see in real participants um you know so yeah so exactly you'd be hoping for is hey

like there's a chunk of people that look like that act as though they have a high risk seeking value and another group of people they have a lower risk seeking value based on their you can fit the model to that explicitly and then you can say hey is the difference in the rs value between these two groups of people does that predict anything interesting you know does that tell you something about you know how well they're going to respond to like drug a versus drug b or does it tell you something about um you know their general cognitive ability or you know their current emotional state or you know like you can do all that kind of stuff which is the ultimate goal of all of this um but so okay so the first thing we need to do um at least in the order i have it set up is to set t so t is the number of time steps in the trial so in this case there are three time steps remember because the uh the agent the agent has to start in the start state and then move to the second time step choose the and then third time step choose one of the machines or it can go straight from the start state to one of the machines but then at the third time step it'll just go back to the start state that's how we have it set up anyway so you need three time steps um so then um what's typically easiest is to first start out by specifying the uh prior expectations over states um and so in the in big d which is the generative process for factor number one which is in the braces here we just specify this as a column vector so a row and then this means that it's transposed and becomes a column this uh apostrophe thing here and so we set this as one and zero which just means that it's with 100 probability gonna be in the left better context and then for the second state factor so this d2 thing here we set it so the agent always starts out in the start state so a one in that first entry and then zeros for all the rest because it's never going to start in the hidden state or in the choose left state or in the choose right state so we're just saying absolutely the agent will always start in the start but then and this is all explained but then for little d this is the agent's actual beliefs we say the agent has no idea what context it's going to be to start with so we set these to values that are flat so where each uh the left better and the right better context have the same values um but notice here that we didn't set these up to add up to one um like we did for big d i mean the reason is is that these are technically what are called dirichlet distributions um and dirichlet distributions um are uh basically they say um the smaller these numbers are the less confident the agent is um in its beliefs so if these are 0.25 and 0.25 it thinks it's a flat distribution but it's really unconfident whereas this thing was like 20 and 20 it would also believe it's a flat distribution but it'd be really really confident that its beliefs are about a flat distribution um and to explain that better um i we're actually going to transition over to chris who's going to give you a little bit of am a kind of more general explanation of how learning works here since you actually do

need to understand this now um so we can transition over to chris here briefly yep because usually a probability people would imagine it sums to one something has to happen you know the coin has to go heads or tails so it has to add up to 100 has to add up to one to be a classical probability but this is a little bit of a tweak on that some similar features but some unique features so thanks for switching over there and helping us understand it yep so can you guys see my screen yes we don't see you but we see your screen all right go ahead all good okay so as ryan was saying before essentially our likelihood what we were talking about before say we're talking about the a matrix there is technically speaking a dirichlet so all of the distributions will be categorical distributions our d's our b's and our a's they're all categorical distributions um and they all have a pro prior or a distribution over the parameter space which is a dirichlet distribution um and so just kind of the people who are technically like in the know as it were uh a categorical distribution is just a special case of a multinomial distribution and a dirichlet distribution is what's called a conjugate prior for the categorical and the multinomial distributions which just means that if you multiply these two things together these two distributions you get out the distribution that has the same form as the prior and that's really important if you're doing like empirical bays for example where the prior or the posterior after round one serves as the prior in round the second round of inference that kind of thing um and so just to briefly go through this so um you guys can see my cursor right yep yeah cool okay this is the categorical distribution where x is some uh categorical variable that occupies one of one decay mutually exclusive states of the world that is to say say i have a distribution over a grid world where my agent is my agent can't be in two places at once it's either in like it's either left or it's right for example um and theta are just the parameters of that distribution and so this function here this thing's sticking out the front is this normalization constant we can adjust this it just counts the combinatorics of the distribution and assume and ensures that sums to one exactly the same thing here with the dirichlet distribution although it is a distribution over the parameter space of our likelihood right and so dirichlet distributions often kind of colloquially called distributions of distributions um because they're a distribution over a vector that has to sum to one and then what's really beautiful about these two distributions is that when you multiply them together and we'll just ignore the normalization constant because things get pretty messy but you end up with a distribution that it has at the same functional form as the prior it's also a dirichlet distribution so our posterior distribution over the parameters of our likelihood is also a dirichlet distribution and all that changes when we go from the prior likelihood is we add to this little alpha thing here it's called a count so we

essentially just add a count to whatever it was that was observed to the variable that was observed so i'm just going to go into a little bit more detail than that on the next slide is there anything you want to add to that ryan um no i mean just just i mean at the at the end of the day and you know chris will chris will cover this you know i'm sure is that while this might look really complicated to somebody who doesn't have a lot of math background it really just amounts um they're being like what i showed you with d there being a number that's assigned to each entry um and when you observe something that number will get bigger um so that means that um like what i said that uh for instance if i believe that i was in the left better state the last five times i'll add like five numbers you know i'll add one five times to that left one so now it would be like 5.25 as opposed to just 0.25 um and ultimately those numbers get soft maxed so they just get normalized and turned back into a probability distribution um but but like i said at the end of the day you're just adding counts each time you observe something yeah this is just like how you might be a little bit more confident about the true ratio of different kinds of colors of balls in an urn if you had three of one color and then one of the other color you're not quite sure if you had three million and one million you'd be very sure and then max you had a question yeah so just going to the other end of the spectrum then if i'm somebody who's in probability theory and i'm talking about a dirichlet distribution and i'm talking about the support for that distribution on the open interval 0 1 and as we can see here from these equations that are provided uh that you know if i if i go to zero then i'm basically going to end up with since it's multiplying each time then there could be problems so in practice however uh my question not being very experienced with using these things in practice when we implement this does that have much of an effect on any of the estimated parameters um no but one thing to say is that because you always work with log you we're always in log space we end up once you just have basically a little numerical trick where all of your to make sure that you never have a log of 0 we add e to the negative 16 to all the entries um does that kind of answer your question yeah so if for example if i see at the end of the day uh i might have a a converged parameter that has apparently a very low log likelihood of uh you know when i when i converge on that estimate is it you know maybe part of that could be related to okay there's some times where i'm entering in a state that might be unlikely to occur most of the time or maybe it wasn't observed in this particular set of experiments but the rest of it kind of holds up is that maybe one way to think about low low uh values uh when i converge to my parameter estimates um i'm not sure i totally understand the example just because it's a bit a little bit abstract could we give it how about the the slot machine example

it's like you go into the casino you've never done any of the slot machines it's like you haven't had much experience so even if you're um not sure about which one's better it's like you're not sure overall and then as you try more of the slot machines just like ryan was saying you increment up observations so to speak in those columns so after you've made a thousand slot machine visits you're a little bit more confident than after you had made zero or one or few is that the case just to bring it to the example that we're gonna be sure yeah or i could pull the slot machine three times and get only one result observed and i would never have um the complement to that result observed and so then effectively my my probability distribution is then one and zero on my posterior um so if i end up having like weird looking numbers in my on my converged parameter estimates in terms of the log likelihoods for the different things it's kind of maybe just relates to this particular property but at the end of the day what it sounds like is that i'm still going to converge to the correct values using this approach that's just like kind of an insurance um yeah i'm not sure honestly about any if there are any formal results that relate to this um but in practice because we always basically the categorical distribution is just and it just ends up being a soft max function over the concentration parameters um you'll always have something that's between zero and one um and the concentration parameters are basically will always kind of be somewhere between zero if it's never been observed and at whatever it is it goes to however high your however many experiments you run or simulations you run um i've never encountered a problem with but then again i've never actually really looked into um possible consequences of that so i just don't know i'm sorry ryan do you ever think so um i mean the only thing i would say yeah and maybe maybe i misunderstood the question but i mean if it has to do with whether or not you'll there's certainly no guarantee that you'll converge on the on the uh on the true generative process yeah there's there's lots of cases where things can go wrong and you'll get stuck in local minima and things like that um we actually show several examples of that in this uh structure learning paper um where we show how you can use how you can use active inference for structure learning and um like bayesian model expansion or state space expansion and safe space production and things like that there's lots of cases where you'll end up with the wrong likelihood um so so it's not uh yeah it's or maybe in my mind just a caveat would then be you know if going to that example with the posterior distribution i only have uh pulled the or picked a machine three times um when we're designing our experiment it might be important to consider setting it up so that you know we're not going to get into the case where we only have a sample of n of three because obviously that doesn't sound like good science to anybody

anyways but it's just kind of the oversimplified case where as you add more categories you could run into that scenario for one of your particular categories which could cause some problems it sounds like if there's a hundred kinds of cereal and you only let people choose one time you're not going to get a good estimate it's kind of like a sample size and an experimental design so just to kind of pull back for those because that's that's a really interesting question about where there's probability or how different sample sizes are related to each other but we're just thinking about this the structure for how additional observations are going to increase the kind or the the quality of this estimate of a parameter yeah and you'll and you'll see and what and what chris shows you that that um like the basically the the lower the concentration val the concentration priority values are generally the more the expected free energy will be weighted toward um choosing like uh parameter exploration policies right so it'll be a thing where if it's never chosen the 99th serial before it'll be driven to go choose that one just because it wants to be more confident in what the actual parameters are but but anyway i uh crystal yeah go go ahead yeah yeah so we actually don't yeah we actually don't have any numerical examples in the paper where we kind of have show the free energy values um for different parameter values we only kind of have state terms but um there are i mean maybe that's something we could add to the paper actually that would be something interesting yeah people yeah okay so essentially just to kind of give a little recap we just have our uh our probability distribution over a or the parameters of a is just a dirichlet over kind of over a and then so we just end up with a matrix that has the number right where the rows are our possible observations and the columns are our hidden states literally all that we're doing this so this little term this little kind of cross inside a circle is called a kronecker tensor product um actually that goodness is that correct i actually don't know that's the notation cross product it's in the spm textbook that notation is used for the chronicler product yeah thank you okay good um sorry i just had a brief mind blank i was like oh god anyway so and this eta thing here is just a learning rate and so whatever it will just multiply essentially this here so say i'll just give an example and we'll just add that to our previous values of whatever these a's are so just very simply say we have our posterior over states is something like 0.7.3 and our op we receive observation so this is transpose vector so this corresponds to row 2. this all that this would do is we're literally adding a value of 0.7 and 0.3 to this matrix that's all that learning is really it's very simple um and then both of these things will get both of these uh columns will get passed through a softmax function at the end of the day and just kind of give a give an example of what we're talking about before so because they're both soft maxed all that soft all soft max

cares about is that the difference between the two numbers in all the two elements of the vector so 51 and 50 and two and one the same same value in the softmax but in one case the model is incredibly confident and in the other case it's not confident at all for example so if you had an agent that was actually doing parameter exploration there would be no the agent would have no motivation whatsoever to explore the parameters in the first case of 5150 but it would be quite motivated in the case of two one for example even though the likelihood distribution you end up getting out of is the same okay good um so that is actually i think all that we really had that's all that i'd prepared for this is because it's really is in practice it's so simple um so back to you ryan okay sorry okay so anyway with that as a little bit of the background always all we're doing here is we're saying to start with the agent believes it's a flat distribution it has no idea which one's going to be better left or right on each trial that's in little d because that's our model so each time it believes that it was in the left better context it's just going to keep adding you know numbers to this 0.25 here and those will be scaled by the learning rate thing that uh that um chris showed where uh you know if if learning rate is one then each time it observes the left better context it'll just add a one so it'll become 1.25 and then 2.25 etc but if the learning rate was like 0.5 so it was like lower then it would add each time so it'd be 0.75 1.25 etc so the learning rate just scales how big the counts are they get added um and the learning rate could be zero at which case there wouldn't be an updating just thinking about just what could the whole range be from not a learning agent to very slowly incorporating new information using this dirichlet framework that christopher just described all the way up to just ridiculously high values so that it just yeah i should just highlight as well i think there's this is i just kind of want to this probably isn't a good point for discussion just to keep things moving but one thing that's worth highlighting is there's a genuine kind of debate or maybe debates a bit too strong word but discussion to be had about whether we should actually be moduling modulating learning rate given that the model is bayes optimal or approximately bayes optimal one would assume that there should be a way of modeling things without actually changing the learning rate from one um but in practice i'm not sure if that actually works but anyway so yeah so i mean i mean technically technically a learning rate of one is optimal yeah but for but for um for when you're fitting behavior when you're fitting the model to behavior um most people are not optimal um and so uh you have to estimate typically the learning rate for each person and oftentimes it's quite a bit below one but that's actually an interesting attribute and again it's an example about how using these generative and generalizable frameworks help us compare not just who could remember more numbers or

who had a faster reaction time as far as just observable characteristics we're moving a step back and asking whether people might differ in their learning rate or in other attributes of this yeah and there's also and again i mean this isn't something to really get into so much but there is effectively a learning rate that comes out of the precision of your posterior beliefs over states um like for instance if your posterior beliefs over states are really imprecise like say it's like point say afterwards you know this is like 0.6.4 then whatever observation that you is just going to add counts of 0.6 and 0.4 which means that you're not really going to build up strong priors um you know uh um a straw a more precise prior for one over the other um because uh you're you're not confident about which state you're in just a quick example of that you go into the casino and you're drawn a card and you can't read the card or it's the number is rubbed off so if there's no precision at the sensory level with which card you drew you're not going to be updating your estimate of how likely different numbers are and then if there's a perfectly legible card so it's 100 you have no error about which card it is there's the secondary question about how much you update your internal model but it's not a precision about the perception question so it's really a nice point and it's worth understanding that you can have uncertainty about observations that then is captured by this model but even in the case of precise or even totally observable scenarios there's additional uncertainty related to learning yeah i mean again there's still potential problems i mean what tends to happen in these models is especially if learning rate is one then like the agent kind of becomes too confident too quickly or or a better way to put it as they once they've like observed a certain number of like lefts for example becomes really hard for it to unlearn that um which is different from what humans do humans tend to infer that there's a different context like okay no now it's now it's a new context where right is better so it doesn't necessarily unlearn the original uh counts it instead sort of infers now i'm in a new state and i need to accrue different counts for that state um and that requires more complex models which can also be built just but couple couple little notes that that's kind of like humans using narrative to help reset local parameter estimations like oh well now it's a new day so now things that were unlikely yesterday now it can be likely today and also it relates to this question about the optimal learning rate in a mathematical or an analytical framework maybe one or some other number is quote optimal but then in a realized ecological setting it might be actually not the best solution in that setting yeah um so i mean yeah typically people talk about this in other literature as being like laden cause inference or leia like inferring the latent causes um behind things there's different latent cause it's kind of the same thing as saying there's a different context and therefore a different likelihood um but uh

but yeah so so i'll just kind of you know go through this here you know kind of step by step so you know so i so i'm gonna set up right so i have um i have my big d and my little d here um and then the next thing i'm going to do is i'm doing this a little bit you know condensed is to just set the number of states here for each state factor so that's going to be the length of d1 and the length of d2 so this one's going to be the two contacts so there's going to be two and this one's going to be the length of d2 which is going to be four states so at the end of the day this ns thing if you look on the right is just going to look like that 2 and 4. just one note there for those who are unfamiliar with matlab so at line 148 where ns is you put a little red dot like a stop sign and then you hit run which ran everything that we had just discussed from a clean slate up to that line and then it halted so that's what allowed us to actually instantiate all these variables we were just talking about and then you stepped just one line at a so that you could run this line 148 with all of its prerequisites fulfilled so it's kind of like a break point in the program we took a breath and then we ran one line and so now we're going to go step by step yeah it just allows you to kind of see what it's doing step by step which is yeah it's helpful so sorry i should have said what plays and step and stuff like that um so so now that i've specified that i can put this little for loop thing which um for that for i equals one to the number of states for factor two um i'm just going to make it all the no hint this is just saying for all the matrices in a1 to start with um everything is just going to generate the no hint observation um so it will end up just looking like this where uh right so i just have so this is just saying um and i should say that the columns correspond to the first day factor so left context right context and these third the third dimension here one two three four that's what corresponds to the second state factor which is the um the behavior so this is saying when i'm in the start state one each is going to generate the no hint observation when i'm in the hint stay factor it's going to generate the no hint observation and so forth so i'm just starting it that way for ease to say that they're all that way but then what i'm going to do is i'm going to define this thing pha which is the probability of the hint being accurate and again this is just kind of to make things uh a little bit concise and convenient and then i'm going to replace that a2 because remember a2 means i'm in the hint state um and so i'm going to say when i'm in column one in column one here i'm gonna observe the machine left hint with some probability if i'm in the machine left state which is the left column and then one minus that for the no hint and again if i'm in the right the right context so the the right column here then it's going to be the reverse i'm going to observe the right machine with some probability just one one note on that ryan so people might look at this and wonder why didn't you just say hint or no hint instead of this one minus one equals

zero and this is because as you've had it it generalizes to cues that are less than a hundred percent accurate so then you could have the probability of the hint being accurate to being 90 or less or some other value and so if you have a probabilistic learning task you can model it with this exact framework just by changing pha equals point something rather than reimagining the entire matrix multiplication which is really the hard part yeah exactly so so if i step through that then in this case i've set pha to 1 which just means the hint is completely accurate so now if i look at a1 again then it's going to look like this where all the other behavioral states so in the start state in choosing the left state or choosing a left machine and choosing the right they those still generate row one here both of both contacts do because the no but if i'm in state two which is the hint behavioral state then if i'm in the left machine or if i'm in the left state then i will observe the left is better hint um with probability one and if i'm in the right column here the right context is better state then i'm going to observe the right hint observation so the bottom row here with 100 probability so in other words if i observe this thing then i'm going to know for sure that i i'm now in the left better um but if again if i made this like 0.8.2.2 then that would just mean you know there's some probability that the hints gonna give me the wrong answer um so that's so that's all that is so remember the first state factor is the columns the second state factor is going to be the third dimension that's kind of key so now what i'm going to do is i'm going to define the second output modality so this is another thing that's important is that the uh the number here for a that's in the braces that corresponds to the outcome modality um so the first set of observations i can get is no hint machine left hint and machine right hint so that's one set of observations that we call like an observation or an outcome modality um now for a two the second observation or outcome modality the are null loss and win um so what this is saying is for the first two behavioral states for i equals one and two um the uh so being in the start state or the hint state is always going to the null observation so it's not going to generate a loss or a win now after that if i step through here i can set what i just called p win so the probability of winning and i set that to be 0.8 and this is the part where the actual reinforcement or reward learning comes in um because what i can do is i can say okay if i'm in the left choose the left machine state so 3 then the probability of winning the probability of losing is 1 minus p win and the probability of winning in the right one or the probability of losing in the right one if i choose the right one is high right which makes sense right because if i'm in the left better then if i choose the right column right if i choose the right machine then i'm most likely going to lose in contrast if i'm in the right better context and i choose uh if i choose the right machine and i'm in the right

better then the right column here should generate a win with high probability this is just saying if i choose the right machine that's the four and i'm in the right con right better context which is the right then i have a high probability of winning and vice versa up here if i choose the left one again just to talk about how this is the minimal example we're talking about two options with the two slot machines there could be more options we're talking about how much it costs to get the hint and how much it costs when you win or how much you receive when you win that's a parameter you can change how accurate the hint is you could have a perfectly accurate hint that sends you to the slot machine that wins 100 of the time so the probability of winning can now be changed so then you can say okay i'm imagining a situation where there's a perfect messenger with a hint and then it's free to visit that person and then it's 100 likely which slot machine is the winning one or you can imagine these more gray zone scenarios where different things are associated with each other in a less direct way or there's more than two options just for those who are like seeing this one example for the first time this is kind of like the tip of the iceberg of a big big family of different kinds of models that have a lot more options and a lot more nuance yeah i mean you could have you know 20 outcome modalities and 10 state factors if you wanted to it would just mean that like there'd be a fourth dimension for the third state factor and a fifth dimension for the fourth state factor and you know it'll just get really big really fast um but so now that i've done this because i set the probability of winning to 2.8 um given that you're you know in the correct context then a2 will look like this so this just says if i choose if i'm in the start state i don't observe a i never observe a win or a loss if i'm in the hit state i never observe a win or a loss um if i'm if i choose the left option the left machine then i will win with point eight and lose with 0.2 and the opposite for if i choose the right one right i'll lose with 0.8 if i if i choose the left machine and i'm in the right context and if i choose the right so state 4 here then the probability of is point eight so the third row here is winning right um and point two and vice versa so this is just defining the probability of given your choice given whether it's the left better or write better context um and i know that's a mouthful but uh yeah um so then finally um and you know this isn't really all that practically interesting but it's important to include is is that what this just says um so outcome modality three is just the agent's own behavior so what this means is every single time it makes a choice it observes itself making that choice and all that does is make the agent completely confident about what it did um but you could imagine if somebody weren't aware of their own decision-making behavior you could imagine that might lead to decision-making that's not in line with what you would do if you could observe your behavior yeah so you know i mean if you if you

get rid of this then in some cases the agent yeah won't be all that confident what he did and so therefore it'll be hard for him to uh hard to learn what actions were the right one because we won't know like what it shows basically uh it'd be like choosing a slot machine blind um i just want to jump in with a question about my motor systems context so for example if i'm studying sensory motor integration and i'm measuring motor cortex signals from neurons and motor cortex and i'm measuring uh outputs such as the kinematics the kinetic forces that are generated and the emg signals in the periphery i could then use that that context to say i could specify it as either one one or a zero one for example if i didn't think that it was a capable observing its past history or its its decision states in the form of some kind of a sensory blockade uh related to my experiments would that be a practical application would does have face validity in this context given that i'm measuring those signals uh one thing that just collect with flag generally speaking in motor control domains you're working with like continuous quantities right right and that's why you use things like kalman filters etc as your models um i'm not sure to what extent a partially observable markov decision process would actually be the appropriate generative model for the situation i think it would be if you want to model like the decision making processes observ for motor control so am i going to move my arm to left or the right that is a discrete decision but actually measuring motor control and a lot of those things that you're talking about those continuous signals i'm not actually sure p o mdp is the but for example if i thought that i had a state space that consisted of a a fixed point or any kind of you know however i'm abstracting those continuous processes into the realm of you know a discrete state uh just glossing over that this kind of model could maybe uh account for that or do you usually use in that way in any i mean in any case you know where where signal is technically continuous i mean you know to use these sorts of models right you're going to have to just pick some quasi-arbitrary or maybe justify in some sense a way to kind of bin you know bin time right you'd have to desperatize time in some in some way um you know the signal was x for these three milliseconds and now it changed to y these next three milliseconds that'd be some kind of like funny average or um i guess i guess the you know potential concern here is also that like like there has to be some way to to link the uh the the states as here as you're kind of bending them to some sort of policy selection process presumably um so no i'm not completely sure at least i can't imagine exactly how that would work it's almost like this one is about the agent deciding which way to walk and then maybe there's some proprioceptive feedback but let's think about what this decision making example is and then wonder about where the motor and where these other modalities and the

continuous signal processing also interesting let's just walk through this discrete time discrete opportunity space model and then we talked about like in active infrastream eight with alec chance about what does it look like to move active inference from a discrete into a continuous space so it's the kinds of things that are actively being developed and it sounds exciting about the motor example max yeah i mean i mean so there are you know and you know thomas parr has done you know a bunch of this like we use mixed models right where the top the top layer is a discrete state space markov decision process model like this you know deciding left arm right arm but then it feeds into a continuous uh state space below um that actually controls the dynamics of the movement itself based on the decision um my guess is it would be more uh something more like that would be better you know that being said these sorts of these sorts of models are much more about you know behavior and choice i mean when you're talking about neural responses i would think that would come more be more useful in these models to to use them to predict the neural responses that you would get when a person does x versus y and see if that matches with the firing rates that you see there's a paper on the mix model thing there's a paper i think it's called like discrete movements to decisions and back again or something along those lines it's in your computation um yeah i think it deals with all the formal issues that kind of arise with yeah thanks about it okay so just for the sake of time here um so so again what i did here and this is for the agent observing their own behavior all i did was i made it like this right so this is just saying when the agent chose uh uh state one then it observes the observation associated with state one right row one with a hundred percent probability when it was in the hint state then it observes that it was in the hidden state row two with a hundred percent probability and so forth so it's just providing maximally precise evidence so that it knows what it did um so now notice that this was big a right so that's the generative if i wanted to if we wanted to do reward learning right if we wanted it to learn p when the what the reward probabilities then we'd have to set little a here um and you might start out just sort of making big a making a little a equivalent to big a um to start with and and uh multiplying each of the values in little a by a big number like 200 and what that does is it just makes it the agent is really really really confident about um its beliefs um um in a in that are associated with big a um and what that does is it just prevents learning for anything you don't want it to learn so if you multiply this thing by really big number then it won't learn anything even though technically it's learning because it's just already too confident um but then what you could do is for anything that you do want it to learn you can just redefine those with really small numbers right so you can say a3 here so the probability of choosing uh of

winning choose if you choose the you know is 0.5 so just 0.5 is all around right or 0.25 or whatever so this is just saying that the agent would start with uh completely imprecise beliefs about whether it would win or lose depending on what it chose depending on so it would so this is what you would do you'd kind of turn on little a if you wanted it to learn the reward probabilities but we're not doing that so we're just showing this as examples and same thing if you wanted to you could do the same thing but for outcome modality one um to have it learn like the accuracy of the hint um so again these are just kind of commented on examples if you wanted to do that kind of thing um so now we've defined the the likelihood matrix right the a matrix like what states generate what outcomes with what probabilities now we're moving on into b which are the transition probabilities and remember that there's one of these per state factor per action um so and this is what i showed before that for b1 so for state factor one the context there's only one action quote unquote and it's just an identity matrix this just says the left better context never changes within a trial um and again i showed this before for the the transitions there's four different possible transitions transitioning from state one to any of the other states so just to be clear these transition matrices the columns are the states you're in now and the rows of the states that you would move to so this is saying from any you could move to state one um you know from any state you could move to state two et cetera um so you end up having you know one two three four different actions for that state factor four different possible transitions so now the next thing that we do is um we move to the preference distribution so that's c right so this is what the wants or what the agent finds rewarding and how rewarding the agent finds it so to do that initially we just with no we just say what are the different number of outcomes for each outcome modality and that just looks at the size of the row dimensions in a so if i do that and again this is just using this you know size function which is just you know again to make it more uh convenient or generalizable so if i do that then no is going to look like that that's just saying i have three i have three different outcome modalities one is no hair null no hint and hint one is null lose and win and one is the agent observing its own behavior right observing action one two three or four um many people have asked in the past weeks about dimensionality of not to go into the markov blanket discussion but the dimensionality of and how we talk about different kinds of outcomes different kinds of senses for example and it's almost looking like it's just about the way it's specified the hint could be in your ear and the observation could be visual or something and so you could get philosophical and ask whether there's really one or two different sensory modalities but in the context of this inferential model that we're writing it's almost like those things don't really

matter they're just variables that are observed yeah i mean so again we call these modality outcome modalities so in this case there's three right so like you said the hint this could be the auditory modality and there's three things that could hear this could be the visual modality and it's the three whether it wins or loses or hasn't seen that yet right this could be visual and this fourth thing the third one could be proprioception right it could be observing or feeling what it did i mean in this case it's probably also somewhat visual right it observes what it does but um you get the point yeah so yeah so there's just a nice factorized you know dimensionality for each modality yeah um but okay so now i've what i've done is i've started out by just putting zeros in the c matrix the preference for each outcome modality so this is just saying for outcome modality one so the hint the agent has no preferences and i should say the the rows or the observations um for the you know null hint no hint i mean the columns are so this is saying at time one the agent doesn't care whether it gets the hint or not in terms of rewardingness um same thing at time two same thing at time three and i've just done that for all three how come modalities to start with the only thing that we want to add preferences for is the win lose observations um so in this case so that's c for outcome modality 2 which is the win loss thing here so what i've done again for generalizability is i've set this parameter la that is called loss of version and i've set this parameter rs which is that reward seeking thing which i just defined above as rs1 and again that's just convenience so i don't have to go down here to reset it so i define these just for again just for convenience and then what i say here is so the observations again are no lose when so this is at time zero the agent doesn't care whether it observes a null a loss or a win and that's just because it can't observe anything but a null at time two it just prefers losing with a value of again negative one here for both time two and time three but here's a little bit of a trick is that i define the preference for winning as rs at time two and remember that rs is four right now because that's what we set it above then at time three i set the preference as rs divided by two um so what that ultimately means is that um the this state factor will look like or i mean sorry this this preference distribution will look like this saying it dislikes losing equally at each time but it wants but it gets more reward for winning at time two than at time three um and so that's our that's our preference distribution um and uh this is just all that's basically it this is all in how the details of how the sketched out scenario okay four dollars but then you got to wait 20 minutes and however your scenario is this is about the detail in matrix form of how the payoffs work and so it could be minus one or you might ask if another number is appropriate there but it's kind of like that's the detail level that's the applications and that's where we hope to see a lot of people exploring different scenarios so

that we can understand better yeah and so and so you'll see that when we actually do the like fitting like parameter fitting to behavior you know one thing we'll do is we'll fit right so for some people rs might be like eight and for other people it might be like two um so you you find this value for each person that best explains or reproduces their behavior um i mean you gave that example last week right in the substance abuse example where that actually has really meaningful or that parameter has a really meaningful interpretation yeah exactly um because it has to do in the case of an explore exploit task the lower rs is basically means the more people the more like someone is driven to explore you know before they kind of jump to conclusions about which one's the best and just keep choosing that one it makes me makes me think about maybe in the um substance example or another example it's like if we're going to have a society that rewards good behavior and punishes negative behavior or something on some issue do you go with here's how much you get positive and here's how much the punishment should be should be small punishment versus large these kinds of questions which are really important at the individual and the group level not that this is even close to the answer but it's almost like it's a way for us to start talking about how much do we value and how much do individuals need different blends of these different attributes so really a good way to talk about this yeah absolutely um you know and so like uh oh so one thing that i did want to clarify here is you'll notice that you know these are supposed to be probability distributions c you know each column here is supposed to be something that adds up to to one and technically we work with log probabilities um so uh you know negative one and four and negative one and two obviously don't uh aren't probabilities or log probabilities um so in practice um these each column here is softmaxed and then each element is logged which is which is how you end up with the actual log probability distributions that define the preferences when they're actually used in the code and i show that i show an example of the numbers they turn into in the in the or we show them in the in the um so you know last kind of really central thing is to define the policies so here i'm just going to use np here to define the number of policies i want to allow and here the nf thing is just the number of actual state factors um and that will just allow me in a generalizable way to say that v has uh t minus one rows right so it's gonna uh the number of actions minus the number of times because you have to move from time one to time two time two to time three so there's always one less action possibility than there times in a trial um and the number of policies again is going to be columns a number of state factors is going to be that third dimension so i can start by just defining that as ones um and i can you know keep it that way i just wrote this explicitly so you

can see it and this just means again it can only choose to stay in the same context every time so that's not something the agent knows or can control whereas for state factor two i can define each of these possible policies stay in the start state take the hint and then choose the left machine you know take the hint choose the right machine choose the left machine and then go back to start choose the right machine and go back to start and i you know i describe what each of those means and this kind of one two three four five thing down here um so then if we want to and i'm not going to do this here um you could specify e which is the kind of like habits you know essentially it's just a pry over policies that biases you towards choosing one thing versus another and it's just one is in in the previous active inference streams we talked about that as the field of affordances for those who are connecting it to the ecological psychology or to the um in activism the prior the prior on what can be done among policies is the affordances of the organism in their niche so if you don't have the object it's not going to be a policy under consideration and so e is this mathematical device that's going to be weighting policies by their habit basically excellence is a habit yeah i mean i i guess like i mean technically what you're doing is you know like the when this makes sense functionally is is when you're learning it so so what you would do with little e um and so this might start out you know flat right with just counts of one for the dirichlet distribution but if you choose the same policy 20 times let's say you choose policy 3 20 times then this one this third one here would become a 20. and so that just means that you'd start out with a strong bias now for choosing policy three because you've chosen it a bunch of times before um and that's um an optimal thing to do under the assumption that option three was the one that continued to have the lowest expected free energy every time right so if in this model based way based on minimizing expected free energy you succeeded over and over again when you chose option three then it makes sense to build up this kind of bias so that you don't actually have to work that hard cognitively to just keep doing the thing that's right once you build up a prior that's going to keep being right so i guess i can see how if you do the same sorts of things in a given niche over and over again then you develop priors that bias you toward doing the things that are successful in that niche and if something's if something's never been if somebody's ever been observed then it's unlikely it takes a different mode of operation outside this script if something's never been observed for it to be considered this is just a basic they know the pop that's why you initiate with a non-zero value because it's something that could happen yeah yeah no that's true if you set a zero anywhere in in e then it's it's as though they don't have that option um yeah that's true um one other thing to say is just like agents can get stuck

if you have non-stationary environments so say you have a hundred trials of one thing and then you switch contexts the agent will probably be stuck um and so i know it's just useful with all of these parameters they have kind of we can have our favorite interpretations related to like ecological psychology or whatever but i think it's always useful to kind of consider them functionally and use them appropriately for this for whatever situation it is you're modeling yeah like that's the thing is you know the the most useful places with this and say like computational psychiatry are if you have the right set of experiences in these models and you get stuck doing what's technically base optimal given your past experience but that like makes you really really maladaptive in what you're doing uh in the new context that you're right so you can get stuck in really maladaptive places in parameter space that could lead to symptoms that look like psychiatric symptoms for example um so i mean this is this kind of idea you know like philip shortenbeck has a paper on this as like you know uh optimal inference and sub-optimal models i think but uh the whole point is you're doing the optimal thing but your model is wrong right um outside the optimal thing under the assumption that your model is right just outside of this model stream which is really amazing and i hope it will continue we would also like to feature these kinds direct discussions about the computational psychiatry so this is awesome with a walkthrough in the code it's going to be helpful for a lot of people who are learning that and then let's table and find the right time to talk about these issues because it's really awesome what you're describing yeah you know i mean obviously computational psychiatry is what i do in practice right so yeah i'd love to um okay so finally there's a bunch of kind of additional parameters right so we talked about learning rate um which here we're just setting to 0.5 for you know arbitrarily um beta this is the expected free energy so it controls essentially how confident the agent is in uh its expected free energy estimates um so if this value is low or sorry if this value is high that means that the agent is really unconfident in its expected free energy estimates and when that's the case then behavior will be a lot more random unless the agent has strong habits if agent has strong habits then having a high beta value means that the habits will have a much stronger influence on on policy uh ultimate policy selection um and we actually show this is the thing that i was talking about it's kind random exploration sort of thing where it's actually updated over time so with each observation the agent essentially updates its confidence in its expected free energy estimates um and then alpha is kind of like a standard what are called inverse temperature parameters where it just kind of controls randomness and action selection so once you've chosen a policy then that policy might say this action is better than that action but occasionally

the agent might still choose a different action kind of randomly um and uh this is actually again this is technically like a sub-optimal thing right you want to just always choose the best thing but this is actually quite important to fit in practice because in actual human behavior there tends to be a notable amount of randomness and so you need to actually fit something like alpha to get a model that fits behavior well now these other two things i'll probably skip over largely because they only have to do with the neural process theory primarily but this erp thing it just controls how much beliefs reset at each time point um with respect to the modeled firing rate changes so it basically controls how much priors at one time point carry over at the next time point with respect to the neural process and then tau is um basically like an evidence accumulation rate so it controls how quickly um you update your beliefs based on new observations um uh as attached to uh the neural process theory um yeah anyone digs into the code it's just a time constant on gradient descent yeah um so and i go through a couple other ones here that i explain but i'm gonna kind skip over those for now so then that's basically it if we wanted to we could hard code what the states what the initial states are and what the observations are like that by setting sno but typically what we will do is we won't do that we'll just let the generative process generate the observations itself which is probabilistic so it'll generate with whatever the probability is of d over d what states are actually the true states in that context and therefore what observations get generated and so last thing and again i already covered this is you just stick all these things you defined into this mdp right and we allow little d which means we're going to allow it to learn whether the left or the right context is more likely we set all these things you know the learning rate and the action precision or inverse temperature so alpha etc we're going to leave out other things we could have defined like e or like learning a or learning b or learning c or learning e etc we're not going to define the states ahead of time and we're not going to define some of these additional parameters that we could include um and then we can just kind of do some uh nice labeling here just to so we label what the contexts uh mean right the semantics we're laying on it for the figures um and that's it and then once we do that then we can use uh little check script here just to make sure we didn't mess anything up in building the model and it'll kind of nicely tell you or give you a hint anyway about what you might have messed up um and then this script that i mentioned before that we provide is the one that actually runs um and then finally once we have these simulations run which will be stored in this big mdp then we can use these plotting scripts actually show the results um and if i set sim equals one then it's going to do that it's just going to do a single trial simulation which i'm going to do now um so i'll just let the thing run the

rest of the way so i won't i'll get rid of these stoppers well just to give a quick let's give a coding recap just to where we are at the end of the second session while you're going through the stop signs so in the first session we talked a lot about the basis of this paper and the rationale we talked a little bit more about who might be interested or what kinds of experiments it was adjacent to and then today that was awesome with the code because we went through a full definition and we moved from that sort of analytical definitions that we went through in the first session into seeing how they're realized in the code and for me at least it was really helpful to see how a lot of these things were put down and then we got all the way up to line 631 or something but it could be different in a little different version but we got to the definition of the full model and then the call out from that object to another script and then we're going to be storing all the outputs in this kind of big container called mdp so today we went from 0 to 60 with the code getting all these run through and defined and operated on now we're going to look at the output and then it sounds like in future sessions we're going to be fitting parameters and maybe doing a few other yeah and future stuff will i mean it sounds like maybe we're going to wait to do to show learning next time i guess depending on the amount of time but um yeah next time then we'd be in a position to do um actual code for learning um we're planning on covering the neural process theory and uh building hierarchical models um and then the last thing is yeah actually fitting models to data um so uh so yeah we'll see how much of that we can get through but um okay so just to finish here so i'm i got rid of all my little stoppers so now i'm just gonna click continue and it's going to run same equals one and what i'm going to get out of this is a little plot like this and also there'll be a neural neural process there you want under here but i'm going to ignore this for now because we're not talking about the neural process theory right now um so what this top one here is is it's hidden states so contexts um now black equals a high probable higher probability white equals a low probability and so there's you know the gray is kind of in between right so this is saying and these are all posteriors over states at the end of a trial so that's important so what this is saying is at the end of the trial at time three these are the beliefs that the agent has about what state it was in across all these time steps so saying at the end of the trial the agent was completely confident that it was in the left better context and these little cyan blue dots here mean that that's what the true context was so in other words the agent was right so this is saying at the end of the trial the thing was the agent was really confident it was in the left better the other state factor is choice states down here and this is just saying that it observed itself go from the start to the hint state and then well sorry it moved from the start

state to the hidden state to the choose left state so those are the states the actions correspond to that well so this is saying it first chose the hint state and then it shows the choose left state or choose left action sorry these are the actions um this thing in the middle on the left it's not really important it's just kind of depicting what the different policy options are kind of arbitrarily based on the numbers so i just ignored that down here at the bottom we have the outcomes and the outcome preference distributions so this is just saying it observed null and then the left hint and then null again this uh the third one just says it observed it go to the start state then to hint state and then to choose left so just observed its own behavior the second one here is the win loss one which is the most important so you can see that this one actually has a non-zero distribution over it which is what means it prefers some observations over others and so in this case what it's saying is it started the null state at time two it stated in the null state right because it took the hint um and then it observed the win um so in other words it got the hint it knew what context it was it chose left chose left and so it observed a win so that's what that means um and then this distribution over policies here is just saying at the beginning it wasn't that confident right so there's a lot of gray across all the policy options but after the hint it became really confident at time two about what the right policy was um this bottom thing is um part of the neural process theory it's simulating um dopamine so dopamine responses quote unquote it's the updates in the expected precision that beta thing um but that's how you read um these plots um and again i won't i won't go over the neural process theory one because we're gonna do that next time hopefully um but just the last thing to show is if you noticed in that case the agent went to the hint first right so it went for the hint first i'll just do that one more time so you can see it's very reliable that it will um that it will choose the hint um with a lot of confidence but now look what happens if i make this rs value higher if i make it eight which means that the thing is uh really really really wants to win the four dollars basically so in this case it should be risk taking it should just forego the hint and just take a guess immediately you can see that's what it does it has a flat distribution has no idea whether it should choose the left one or but it just takes a chance because it really wants to win the four dollars and value of information and the expected free energy is low because the reward value is really high and so it just takes a guess and it guesses right and it turns out that it was in the left context so it observes that it lost um at time two and because it observed that it lost in the first o i must have been in the left context as opposed to the right one so at the end it still knows now that it was in the left better context so that's so that's what that means you can see right away that by changing how much the thing values reward in the changes how

information seeking it is versus how risk seeking it is um so that's uh so that's the way that that works um so uh do we want to end here or do we want or is there time to show learning um we can absolutely go through learning if you would like so the topics that i've written down i have learning which it sounds like would be awesome to run through now then we have actual data hierarchical neural process theories and then so that's the stack of topics let's get learning today and then another time another model stream okay sounds good so learning um so if you could see i sort of said how to do this here i said for to reproduce figure seven so for figure nine here which is sim2 which is the initial learning i'm gonna first set uh rs to three um it's just a value that turns out it's nice for showing the dynamics of learning in a way that's clear um so because it should start out being information seeking and what we want to show is how it learns to be more uh confident and therefore starts to forego the hint as it builds up a stronger prior that the left context is the one that keeps being the case so now what i'll do is i'll set sum equals two but i'll show you what this does down when i go to same equals two um okay so this is actually incredibly simple um so when sem equals two is set all i do is i say okay i want 30 trials so n equals 30. um to keep the mdp thing something that um remains in that just kind of single structure um i redefine little mvp as big mdp and again this is just for convenience and then i can use this uh deal function which is just kind of a nice convenient function in matlab which more or less just says repeat the mdp structure 30 times so now there's mdp1 mdb2 mdp3 and all those are just the same just identical as the initial mdp that we built um so if i do that then what will happen is it will look like so okay so i'll do so i'll do two things one is so before um so now i just have big mdp equals mdp so now big mdp will look like this right there's just a single field for t v a b blah blah blah right everything that we set um now once i run this deal thing then if i step then all of a sudden mdp will now look like this um where you can see now that same is repeated uh in 30 rows and this will all start out being identical so what will happen now though is if i run it through the vbx tutorial script then and this will take a second it will run through the whole thing 30 times and again this will just take a sec so wait let me ask something so first maybe someone's maybe has a little noise in their background but it seems like this little mdp is like a column that conveys all the details for a single trial and then we're making a meta matrix we're just generalizing the matrix into many dimensions and now we're stacking all of the total model as like a column into in this case 30 trials but you could do 100 trials in a row or you could have 20 sets of 10 trials so it's the idea of concatenating total model setups like a column so that you can do operations on big big potentially hierarchical sets of in this keys each mdp is a row really it's just a giant model

yeah it's just a giant structure with gp over and over again yeah i meant bro sorry good good call my r centrism i'm always like thinking in yeah all right but okay so now i ran it i ran the mdp with all those different repeated repeated rows repeated models through this mdp script here and now what you'll see is now the mdp structure will look a little different so now in addition to how the stuff in had before it's also now going to have um all this this thing is like bin now it's going to have all this other stuff it's going to have the free energy f it's going to have the expected free energy g it's going to have the total free energy h it's going to have the actions u it's gonna have the free energy for the uh learned parameters d um a bunch of other stuff and we have a whole table in the in the tutorial that says what each of these fields mean so you can interpret them they don't all matter enough for me to go through them right now um but so then once i have that then i can generate a plot with this little game underscore tutorial script that i for plotting multiple trials that we put and i can run that and it will generate a multi-trial behavior um so so what this is showing is the first action on each trial um so here what you can see is that in the start and the colors are the probability so dark equals higher probability and the the blue circles are the actual chosen actions um so you can see for the first several trials the agent kept taking the hint but after a while because the left one if the left context kept being the one the the one that it thought it was in at the end it starts to be confident around trial 9 10 here that it's always just gonna be the left one so it just immediately chooses the uh the left arm or the left machine um but now what you can see is it gets one wrong right because the it's only eighty percent right that the left one is going to be the right one um and uh once um could a dog be metered or is that yours yes it is it is a dog wanting to want to be in my lap if the dog is a contributor then absolutely welcome to speak yes but uh but anyway so it starts to get one wrong here around trial 12 right because 13 because again it's only even when it's in the left better context it still will lose 20 of the time right and so once it loses it says okay actually i'm not confident anymore so it starts taking the hint again and that happens a couple of times um and so it starts kind of going uh bouncing around um and also note that this uh this is under um an alpha value right like the action precision value that's not that high so its behavior is a little random right there's a little bit of randomness in it like what a real human would do um and so this next column this next plot here below is just green is a win black is a loss and the um you can see when it loses the uh negative free energies are larger and again this i won't go through these last uh these two here just part of the neural process theory they're like the expected uh erps and expected dopamine responses the model that the theory would predict and then this bottom one is just kind of the

evolution of the agent's priors about whether it's going to be the left wing or the right-wing context um so that's how to read this so it learns and then but look what happens if i instead make the thing a little bit more uh risk seeking so i make the the value of the the subjective value of winning four dollars more um and i do that just by setting rs to four instead of three in this case um so now by the way it's awesome that basically by hitting play on this script it seems like even with minimal tweaking people can reproduce this in their own setup start to play with some of the variables and it's just really an awesome toolkit this is pretty cool yeah no thank you yeah we we definitely tried to make it as easy as uh user-friendly as possible for you know people who have minimal background so i'm glad if you guys think um but okay so so in this case right i set the uh risk seeking parameter and the preference distribution to be a higher number um so now if you look the agent is now it takes the hint twice and then just sticks with the left every single time even if it loses a couple of times it just continues to be risk seeking um and again everything else is basically the same except the um the predicted dopamine responses and erps and things like that are different um but but so that is what it looks like when you just do like a normal learning um just where it says the same the context is the same across trials and the last thing i'll show you guys is if i set it to three then we engineered this kind of this reward learning um or this reversal learning version right so reversal learning is for the first several trials the left context is so it's going to build up a prior that the left context is better but then it's going to switch to the right context being better and we're going to see how the agent does now i'll show you how we do that which is also very simple here we're going to set n to be 32 trials do the same kind of you know deal thing but now what i did is i just said for one two uh the number of trials divided by eight we're going to make big d one zero this is saying the true neural process or the true degenerative process is going to be the left one but then i say in the mdp for n divided by eight plus one day plus one to n so basically one after this number to the end then it's going to now be switched to the generative process as the right context so in this case it just means the fir the first eight trials are left better and the rest of the trials up to 32 are going to be the so that's literally it then you just run the same thing through the same vbx function and you plot it with the same plotting script so that's really very very simple so if i do that with risk seeking of 3 then what we'll see happen is um then you know give it a second to cook here just to give one though on that software design instead of having five scripts called simulation one simulation two simulation it's that there's a common core and then scenarios are defined basically like down here in the 600s lines and then by changing which simulation is in play at the very top it seems like ryan

is being able to change which simulation we're in very uh fluidly so we can change the simulation look at a variable change a variable go back up change the simulation go back in yeah i mean it's basically it's just it's just an if then function it just says if same equals three then do this or else if sem equals four do yeah i mean it's just a very simple if-then statement um but uh but okay so in this case um so i set rs equals three um and what you can see is is in this case the agent just continued to take the hint the entire time um because it was never it was never confident enough and the uh you know it happened to get the first one wrong in this case which is part of what drove that um but so if the if the actual observations i got were a little different let's actually try that again and see uh what um what i would get here but this characterizes all these interesting things like a um perhaps over or under eagerness to ask for hints or acquire more information it's showing how the model this one is not trying to fit human behavior we're really playing with the bare bones but already we're seeing that's kind of pathological hint behavior but it never goes when it knows and so now something else yeah well it's just it's parameter play but it's going to play out differently yeah so like in this case where it does take the hint every time it does kind of start you can see on trial five here it starts to have a little bit of like ah maybe i should just choose the left choose the left one immediately but then it gets a couple wrong and then the context changes you can see in context learning here it starts out oh i think the left one's better and then it starts switching and then around trial 11 and 12 now it starts to become more and more confident that it's in the right win context so now last thing i'll show you is what happens under that same but when i set rs equal to four so it's a little more risk seeking um because this actually gets a little more fun so the risk seeking is like a multiplication on the money so it's like one to two dollars versus one million to two million like how high the stakes are because it sounds like you're kind of tuning up whether the agent wants the big wins or what is it really conveying in this model it's just it's just essentially the precision of the preference distribution so it's just it's just how big the number is uh over win at time step two uh but that's really all it is it's just how high the probability is for a um in the agent's model which just encodes how strongly they prefer winning um you could think about it as like four dollars for them subjectively being like the value of ten dollars or something like that but but all it ends up doing in practice is it ends up saying the uh reward value component of expected free energy has a higher weight than the information value component um is is all it ends up being in practice um so here you can see that the behavior is actually you know again it's more interesting so you can see it takes a hint twice and all of a sudden it's like all right i'm confident enough i'm going to go for choosing

left immediately right it does that for a few trials and then the context switches you know so it trilate it switches to being the right context and then it's like oh nope i'm going to start taking the hint again and then it observes the right one uh a bunch of times and then it's like okay now i'm confident it's the right and then it starts choosing the right one without taking a hit a bunch of times now the behavior is quite a bit more interesting because it becomes confident it loses confidence and then it switches to being confident again in the other it gets one wrong around 23 it gets a wrong one and then it does one hint and it's like nope i'm on the right track i just lost one time it's all good yep exactly um so so that's that's the way the learning works and if i show you you know like i mean i'm plotting this here but like literally in the mvp like it will just be if i go to the things learning d right so if i go to little d trial one then you know it observes you know it's 0.75.25 because it observed left the first time right and then at you know say like trial eight it's two point two two point two uh so it didn't make it's because that's what it's considered it was flat still but that's because it got something wrong that's when it was confused yeah and then but if i go down to say you know at the end say like trial i don't know 31 or something then all of a sudden um it's 2 versus 13 right so it's way more confident it's added a lot more counts it being the right context that's better um and that's you know that's really it so and that's literally all the learning is and um you know like uh uh chris showed you the equation for learning um a but you know in the um in the tutorial we also show it for um for learning d which is even simpler it's just uh see if i can get to it real quick here learning process theory learning uh yeah it's literally just so d for the next trial is d for the previous trial plus eta times whatever the posterior states were for the previous trial um that's you know that's literally it um although i'm realizing uh that should be a minus not an equals i think but um a little typos here and there um but anyway that's you know that's really it um so it's not uh it's not complicated at all it's just add account based on what your posterior was over states at the end of the last trial um so i mean that's you know if we're stopping at the end of of basic learning then then that's um uh like i said four and five have to do with estimating parameters based on data and then recovering parameter estimates and fitting behavior to data and things like that so we'll cover that at another time awesome well just to um catch our breath at the end of this really fascinating and very appreciated stream and if anyone wants to put any last comments in the live chat they'll have just a couple minutes to do so let's just each take a final recap or a final thought what do we want to remember from this time what are we gonna take forward into next time what are we looking forward to learning more about so whoever wants to go first maybe christopher any remarks on ryan's presentation and then on the code and also

like your role or which parts had you done i was just curious about that with all those things for the last um so no i think i agreed with everything that was said there uh one thing to just say very when when you really get your hand get a handle on building these models it will take i think for the most time i've ever spent building these models once we'd actually figured out the model structure was a couple of days like you you might spend a very very long time figuring out the model structure but once you've figured out the model structure everything falls in place and it's super easy um so it's kind of interesting in the sense that i have some friends do like neural network or modeling with like spiking neural nets and they can spend like weeks building the the hard work here is really conceptual um and then do you ask like what part of the tutorial i wrote or versus ryan um was that the question not a uh partitioning just which parts were your background drawing from or just how how do you see similar differently i wrote them generally speaking i wrote the mathematical appendices and then did first drafts of a lot of the techie sections but that was kind of like my i really really asked ryan to do that just because i wanted to kind of get a better understanding of myself um and send ryan kind of wrote a lot of the broader sections that was kind of how that stuff worked out um yeah i mean i mean yeah i wrote i mean i wrote the the code we're going through now was was you know i took the first pass of it um chris wrote uh the um the hierarchical modeling code um which uh another another day you'll see in a lot of the neural process uh simulation uh code um or the kind of custom one uh the the ones i was showing you now are just kind of like carl standard the ones that are in spm but uh chris made some way way cooler ones for the the hierarchical neural process simulation stuff um so um but yeah i mean a lot of this was fully collaborative you know i wouldn't necessarily say you know one person did anything more than the other um so yeah every anyone who's like written a scientific paper knows like it's after the first draft has been written it's kind of hard to tell the finished product because when you have heavy backwood and fold editing it yeah i totally agree it's a great perspective so also thanks for sharing that so maybe now let's have our last little round maybe max or we'll each take a last step yeah i'm just uh you know to as a lead into next week i'm excited to hear about the neural process theory and about you know looking at those dopamine traces and it's cool to see the spiking that you are able to reproduce with this kind of a model structure that does reflect you know kind of how the exact phone neurobiological processes might look um one thing i'm really keen to hear about is um the you know the link function between your your gradient um on free energy and your dopamine expression or simulated dopamine expression levels and the the postulated link between um you know precision insofar as i mean

i mean these are things that i'm just it's a little out of my realm but um i think it's really interesting because fundamentally that's that's such an elegant uh mechanism if it's that gradient only um from a mathematical standpoint that it's really cool to tie that from the math and theory into empirical observation and using that to validate cool christopher definitely we're excited we're excited to show you nice do either of you want to make any last comments no uh this has been really fun thanks daniel yeah yeah yeah thanks everyone talking this whole time so yeah people probably don't want to hear me ramble anymore not till next time at least but yeah thanks everyone for watching live and in replay and everything and just for participating because it's the conversations we're having the real time errors we're finding and the learning that we're all contributing to so just thanks everyone for participating and we'll see you later for